Universidad Metropolitana

Departamento de Gestión de Proyectos y Sistemas

Sistemas Operativos

Trimestre: 2526-2

Preparadores: Sofía León y Marielena Ginez ❤ .

Proyecto 2: Simulador Virtual de Sistema de Archivos Concurrente con Gestión de Permisos, Asignación de Bloques y Recuperación ante Fallos

Planteamiento del Problema

El objetivo de este proyecto es que los estudiantes desarrollen un simulador avanzado de sistema de archivos en el que puedan comprender y aplicar conceptos fundamentales como la gestión de archivos y directorios, la asignación encadenada de bloques de almacenamiento, la administración de permisos, la fragmentación del espacio en disco, el manejo de operaciones de entrada/salida mediante procesos de usuario, y el control de concurrencia en el acceso a archivos compartidos.

Para ello, los estudiantes deberán implementar un sistema de archivos simulado en Java utilizando NetBeans, con una interfaz gráfica que represente visualmente la estructura jerárquica de directorios y archivos mediante un JTree, así como la distribución de bloques en una Simulación de un Disco (SD), una tabla de asignación de archivos, y un sistema de gestión de procesos que realicen operaciones de E/S.

El sistema debe operar en dos modos de usuario: modo administrador y modo usuario. En el modo administrador se permite realizar todas las operaciones, incluyendo crear, modificar y eliminar archivos y directorios, gestionar los procesos del sistema, cambiar las políticas de planificación del disco, y visualizar información completa del SD. Por otro lado, el modo usuario restringe las acciones a solo lectura de archivos propios o públicos y la creación de procesos para realizar operaciones de E/S sobre sus propios archivos, impidiendo modificar archivos del sistema o acceder a información de otros usuarios.

Los archivos creados deberán tener un tamaño en bloques, los cuales serán asignados utilizando el método de asignación encadenada, en donde cada archivo se representa como una lista enlazada de bloques en el SD. Sin embargo, la asignación de estos bloques no se realizará de manera directa, sino que será gestionada por procesos de usuario que soliciten operaciones de E/S al sistema. Cada vez que un proceso necesite crear, leer, actualizar o eliminar un archivo, generará una solicitud de E/S que será procesada por el sistema de archivos según la política de planificación de disco activa.

El sistema deberá mantener una cola de procesos, donde cada proceso tendrá un estado (nuevo, listo, ejecutando, bloqueado o terminado) y estará asociado a una operación específica sobre el sistema de archivos. Cuando un proceso realiza una solicitud CRUD, esta solicitud entra en la cola de E/S del disco, donde el planificador determinará el orden en que serán atendidas las solicitudes según la política configurada (FIFO, SSTF, SCAN, C-SCAN, entre otras).

Adicionalmente, el sistema deberá implementar control de concurrencia sobre archivos compartidos mediante locks, con el fin de simular un acceso realista multi-proceso al

sistema de archivos. Se deberá permitir el uso de lock compartido para lectura y lock exclusivo para escritura, bloqueando automáticamente procesos que intenten acceder a un recurso ocupado y liberando locks al finalizar cada operación.

De forma adicional, el sistema puede implementar un mecanismo de journaling inspirado en sistemas reales como NTFS, registrando operaciones críticas sobre la estructura del sistema antes de ejecutarlas, permitiendo simular recuperación ante fallos y garantizando la consistencia del sistema.

La interfaz gráfica debe mostrar en tiempo real el estado del sistema: la estructura de directorios y archivos en el JTree, la visualización del disco con los bloques ocupados y libres (indicando qué proceso o archivo ocupa cada bloque), la tabla de asignación de archivos, y una vista de la cola de procesos con sus estados actuales y las operaciones que están solicitando. Además, deberá incluir una visualización clara de los locks activos por archivo, el estado del buffer (si se implementa), el estado del journal, y un log de eventos del sistema.

Además, el sistema deberá actualizarse en tiempo real cada vez que se realice una operación CRUD (Crear, Leer, Actualizar, Eliminar), reflejando los cambios en la estructura de directorios, en el estado del disco SD, en la tabla de asignación de archivos, en el estado de los procesos, en los locks activos y en las métricas del sistema.

Requerimientos Funcionales

1. Visualización de la estructura del sistema de archivos

- Implementar un JTree para representar la estructura jerárquica de directorios y archivos.
- Mostrar información del archivo o directorio seleccionado (nombre, tamaño en bloques y dueño).
- Referencia de aprendizaje: Pueden echarle un ojo a este video que describe cómo utilizar este componente Curso Java Anexo II. JTree. Vídeo 266

2. Simulación del SD y asignación de bloques

- Representar visualmente el SD como un conjunto de bloques, indicando cuáles están ocupados y cuáles están libres. Se deben diferenciar los archivos almacenados mediante el uso de distintos colores.
- Simular la asignación encadenada, donde cada archivo se almacena como una lista enlazada de bloques.
- Manejar la liberación de bloques cuando se eliminan archivos.

- Definir un tamaño limitado de almacenamiento, evitando la creación de archivos si no hay espacio disponible.

3. Gestión de archivos y directorios (CRUD)

- **Crear:**
  - Los administradores podrán crear **archivos y directorios**.
  - Se especificará el tamaño del archivo en bloques, los cuales serán asignados en el SD.
- **Leer:**
  - Todos los usuarios podrán visualizar la estructura del sistema y sus propiedades.
- **Actualizar:**
  - Solo los administradores podrán modificar el **nombre**
- **Eliminar:**
  - Al borrar un archivo, se liberarán los bloques asignados en el SD.
  - Al eliminar un directorio, también se deben eliminar todos sus archivos y subdirectorios.

4. Planificación de disco

- El planificador debe determinar el orden en que serán atendidas las solicitudes en la cola de E/S según la política que se seleccione en la interfaz, por ejemplo: FIFO, SSTF, SCAN, C-SCAN, entre otras.
- Se deben configurar al menos cuatro (4) políticas.
- La interfaz gráfica deberá mostrar la posición actual del cabezal del disco y su desplazamiento en tiempo real durante la ejecución de las políticas de planificación.
- Para fines de demostración, el sistema permitirá definir la ubicación inicial del cabezal de manera arbitraria al inicio de la simulación.

5. Modo Administrador vs. Modo Usuario

- Implementar dos modos de uso:
  - Administrador: Permite realizar todas las operaciones.
  - Usuario: Restringido a solo lectura.
- El modo se seleccionará mediante la interfaz.

6. Tabla de asignación de archivos

- Implementar un JTable que muestre:
  - El nombre del archivo.
  - La cantidad de bloques asignados.
  - La dirección del primer bloque.

    - Si van a usar colores para representar a los archivos, entonces por favor incluyan el color correspondiente al archivo
- La tabla debe actualizarse en tiempo real con cada operación CRUD.

7. Almacenar el estado de los archivos en el sistema

- Los estudiantes podrán elegir almacenar la información del sistema de archivos en un archivo JSON para que los datos puedan ser cargados en futuras ejecuciones.

8. Recuperación ante fallos con Journaling

- Implementar un Journal (log de transacciones) para operaciones críticas (crear, eliminar).
- Antes de ejecutar una operación crítica:
    - Registrar en el Journal la operación como PENDIENTE.
    - Ejecutar la operación en el sistema.
    - Marcar la entrada como CONFIRMADA (commit).
- Incluir un mecanismo de “Simular fallo” que interrumpa el sistema en un punto intermedio de la operación (antes del commit), para evidenciar el uso del Journal.
- Al reiniciar:
    - Revisar el Journal
    - Deshacer (undo) operaciones pendientes no confirmadas para restaurar consistencia.
- Mostrar el Journal en la interfaz para que sea verificable durante la defensa.

Casos de Prueba Recomendados

Políticas de Planificación

Estos casos de prueba permiten validar el comportamiento de las políticas de planificación. Se sugiere realizar estos CP, ya que las preparadoras llevarán a cabo un proceso similar en la defensa para comprobar el funcionamiento correcto de la simulación. Se deben cargar previamente los System Files y dejar el cabezal en la posición indicada para hacer las pruebas. Posteriormente, se debe permitir escoger la política de planificación.

Nota: Es importante que prueben con casos diferentes. Códigos *hardcodeados* serán penalizados.

1. Caso P1 – FIFO

Elemento	Valor
Cabezal inicial	50
Solicitudes	95, 180, 34, 119, 11, 123, 62, 64
Política	FIFO
Orden esperado	95 → 180 → 34 → 119 → 11 → 123 → 62 → 64

2. Caso P2 – SSTF

Elemento	Valor
Cabezal inicial	50
Solicitudes	95, 180, 34, 119, 11, 123, 62, 64
Política	SSTF
Orden esperado	62 → 64 → 34 → 11 → 95 → 119 → 123 → 180

3. Caso P3 – SCAN

Elemento	Valor

Cabezal inicial	50
Solicitudes	95, 180, 34, 119, 11, 123, 62, 64
Dirección	↑
Política	SCAN
Orden esperado	62 → 64 → 95 → 119 → 123 → 180 → 34 → 11

4. Caso P4 – C-SCAN

Elemento	Valor
Cabezal inicial	50
Dirección	↑
Política	C-SCAN
Orden esperado	62 → 64 → 95 → 119 → 123 → 180 → 11 → 34

Journaling

1. Caso J1 – Crash en CREATE

Elemento	Valor
Operación	CREATE A.txt (4 bloques)
Punto de fallo	Después de asignar bloques, antes del commit
Resultado esperado	Archivo no aparece y bloques se liberan tras reinicio.
Evidencia	Journal PENDIENTE → UNDO aplicado.

Por ejemplo, un Json siguiendo la estructura a continuación debe poder cargarse para poder ejecutar pruebas.

```JSON
{
  "test_id": "P1",
  "initial_head": 50,
  "requests": [
    {"pos": 11, "op": "READ"},
    {"pos": 34, "op": "READ"},
    {"pos": 62, "op": "UPDATE"},
    {"pos": 70, "op": "READ"},
    {"pos": 95, "op": "UPDATE"},
    {"pos": 119, "op": "DELETE"},
    {"pos": 131, "op": "UPDATE"},
```

```
    {"pos": 180, "op": "READ"}
  ],
  "system_files": {
    "11": {
      "name": "boot_sect.bin",
      "blocks": 2
    },
    "34": {
      "name": "readme.txt",
      "blocks": 1
    },
    "62": {
      "name": "script.py",
      "blocks": 8
    },
    "70": {
      "name": "style.css",
      "blocks": 6
    },
    "95": {
      "name": "config.sys",
      "blocks": 4
```

```
    },
    "119": {
      "name": "image_01.png",
      "blocks": 12
    },
    "131": {
      "name": "data_log.csv",
      "blocks": 28
    },
    "180": {
      "name": "video_clip.mp4",
      "blocks": 52
    }
  }
}
```

Nota: Tienen libertad en cómo aplicar las operaciones update en este tipo de casos automatizados.

Consideraciones Técnicas y Reglas de Entrega

- **Conformación de Equipos:**
  - El proyecto se realizará en parejas (máximo 2 personas). Excepcionalmente, se permitirán grupos de 3 solo si algún estudiante queda sin pareja.
  - No se permiten equipos conformados por estudiantes de diferentes secciones.
- **Tecnología y Entorno:**
  - Lenguaje: Java (Versión 21 o superior).

    - IDE: Estrictamente en NetBeans. Los programas que no se ejecuten adecuadamente en este entorno serán calificados con 0 (cero).
    - Restricción de Librerías: Sólo se permite el uso de librerías externas para la visualización de gráficas (ej. JFreeChart), el manejo de archivos JSON, y el uso de Hilos y Semáforos.
    - Estructuras de Datos: Queda terminantemente prohibido el uso de java.util.ArrayList, Queue, Stack, Vector o cualquier colección del framework de Java. Los estudiantes deben programar sus propias estructuras (listas enlazadas, colas, etc.) para gestionar los procesos y el PCB.
- **Estándares de Desarrollo en GitHub (Obligatorio):**
    - El uso de un repositorio en GitHub es obligatorio. Proyecto sin repositorio será calificado con 0 (cero).
    - Uso de Ramas (Branches): No se permite trabajar únicamente en la rama main. Se debe evidenciar el uso de ramas por funcionalidad (ej: feat/scheduler, feat/gui, fix/interrupts), y se debe contar con una rama develop.
    - Gestión de Tareas (Issues): El equipo debe registrar las tareas pendientes y errores encontrados mediante el sistema de *Issues* de GitHub.
    - Integración (Pull Requests): La fusión de código entre ramas debe realizarse mediante *Pull Requests* comentados, simulando un entorno de trabajo profesional.
    - Contribución equitativa: El historial de *commits* debe reflejar una participación equilibrada de ambos integrantes. Un desbalance significativo en los aportes afectará la nota individual.
    - Commits: Los mensajes deben ser descriptivos (ej: fix: corrige puntero en cola de listos-suspendidos en lugar de update code) y el tamaño debe ser limitado.
- **Documentación e Informe:**
    - Se debe entregar un informe en formato PDF que incluya:
        - Descripción de los métodos más importantes.
        - Explicación de la implementación de los hilos para el manejo de interrupciones.
        - Análisis Comparativo: Conclusiones sobre el rendimiento de cada algoritmo (FIFO, SSTF, SCAN, C-SCAN).
    - No es necesario documentar cada línea de código, pero sí la lógica de los algoritmos y la transición de estados.
- **Interfaz Gráfica (GUI):**
    - Es un requisito indispensable. Proyectos sin interfaz gráfica o que solo funcionen por consola serán calificados con 0 (cero).

    - La interfaz debe ser intuitiva y permitir visualizar el cambio de estados del proceso en tiempo real.
    - Es obligatorio implementar validaciones de tipo de dato y rango en todos los campos de entrada. El sistema debe ser capaz de gestionar entradas inválidas sin interrumpir el flujo del simulador.
- **Entrega y Evaluación:**
    - Fecha límite: Lunes de Semana 12 antes de las 7:00 AM.
    - Canal: Enviar informe PDF y link del repositorio a: sleon@correo.unimet.edu.ve y mginez@correo.unimet.edu.ve.
    - Asignación Presencial (Defensa): Se realizará el Lunes de Semana 12. Es individual y obligatoria.
        - Si un estudiante no asiste: 0 (cero) en el proyecto.
        - Si un estudiante reprueba la defensa: Su nota máxima será de 10 (diez) puntos, independientemente de la calidad del código.
    - Es imperativo que ambos miembros dominen el funcionamiento total de la solución, no solo su parte asignada